

TP1 – Acquisition et Nettoyage des Données

M1 Data & IA – Université Catholique de Lille

Rendu attendu

- Un notebook Jupyter `.ipynb` exécutable de bout en bout.
- Travail en binôme conseillé.
- Pour chaque question, la réponse doit apparaître dans une cellule **Markdown** placée sous le code.
- Nom du fichier : `TP1_DVF_NOM_Prenom.ipynb`

Contexte

Vous êtes recrutés comme *data analysts* par une foncière implantée à Lille. Votre responsable vous confie un extrait de la base **DVF** (*Demandes de Valeurs Foncières*), qui recense les transactions immobilières en France. Votre mission consiste à analyser les ventes immobilières du **département du Nord (59)** pour l'année **2023**.

Ce TP prolonge directement le cours : vous allez charger un jeu de données réel, l'explorer avec **Pandas**, évaluer sa qualité selon plusieurs critères, comparer deux formats de stockage, puis organiser votre traitement sous la forme d'un pipeline **ETL** clair et réutilisable.

Jeu de données

- **Nom** : Transactions immobilières DVF – département 59 – année 2023
- **Source** : <https://files.data.gouv.fr/geo-dvf/latest/csv/2023/departements/59.csv.gz>
- **Format** : fichier CSV compressé `.gz`
- **Séparateur** : `,`
- **Volume** : environ 30 colonnes

Objectifs pédagogiques

À l'issue du TP, vous devez être capables de :

- charger et inspecter un jeu de données réel ;
- manipuler un **DataFrame** avec **Pandas** ;
- identifier des problèmes de qualité de données ;
- produire des indicateurs simples d'analyse exploratoire ;
- structurer un traitement sous forme de pipeline **extract / transform / load** ;
- justifier des choix techniques dans un notebook clair et documenté.

1. Mise en place

1.1. Environnement Python

Installez les bibliothèques nécessaires avant de commencer.

```
pip install numpy pandas matplotlib seaborn ydata-profiling jupyter pyarrow
```

1.2. Imports et configuration

Créez dès le départ la structure de votre notebook avec les titres suivants :

```
# TP1 - Exploration et pipeline ETL - DVF Nord 2023
## 0. Imports et configuration
## 1. Chargement et inspection
## 2. Analyse exploratoire
## 3. Diagnostic qualit
## 4. Structuration du pipeline
## 5. Synthse
```

Ajoutez ensuite les imports de base :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import time
import os

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", "{:,.2f}".format)
```

1.3. Chargement du fichier

Utilisez l'URL officielle du département 59.

```
url = "https://files.data.gouv.fr/geo-dvf/latest/csv/2023/departements/59.csv.gz"
df = pd.read_csv(url, sep=";", low_memory=False)
```

2. Chargement et inspection

Dans cette partie, vous prenez en main le jeu de données et vous vérifiez que les manipulations Pandas reposent bien sur les performances de NumPy.

2.1. NumPy et Pandas

Le cours a montré que NumPy est plus rapide que des boucles Python classiques grâce à des opérations vectorisées. Vérifiez-le expérimentalement.

```
import numpy as np
import time

n = 5_000_000
data = list(range(n))
arr = np.array(data)

# Boucle Python
t0 = time.time()
_ = [x * 2 for x in data]
t_python = time.time() - t0
```

```
# Operation vectorise NumPy
t0 = time.time()
_ = arr * 2
t_numpy = time.time() - t0

print(f"Python : {t_python:.3f} s")
print(f"NumPy : {t_numpy:.3f} s")
print(f"Rapport : NumPy est x{t_python / t_numpy:.0f} plus rapide")
```

Question 1. Quel rapport de vitesse observez-vous ? Expliquez pourquoi **Pandas** bénéficie du même avantage lorsqu'on écrit une opération du type `df["colonne"] * 2` plutôt qu'une boucle `for`.

2.2. Inspection initiale du dataset

Question 2. Chargez le dataset puis affichez :

- le nombre de lignes et de colonnes ;
- la liste des colonnes ;
- les cinq premières lignes.

Question 3. Inspectez les types des colonnes. Repérez au moins deux colonnes dont le type n'est pas adapté, puis corrigez-les.

Indication : cherchez notamment :

- une date encore stockée comme `object` ;
- un identifiant interprété comme nombre alors qu'il ne sert pas à faire des calculs.

Question 4. Calculez les statistiques descriptives des variables numériques. Que vous apprend l'écart éventuel entre la moyenne et la médiane de `valeur_fonciere` ?

3. Analyse exploratoire

L'objectif de cette partie est de fournir des éléments de lecture simples du marché immobilier du Nord en 2023.

3.1. Analyse des prix

Question 5. Filtrez les **appartements** puis calculez :

- le prix médian ;
- le prix moyen ;
- le nombre de transactions.

Commentez les résultats dans une cellule **Markdown**.

Question 6. Créez une colonne `prix_m2` définie par :

$$\text{prix_m2} = \frac{\text{valeur_fonciere}}{\text{surface_reelle_bati}}$$

Affichez ensuite les statistiques descriptives de cette variable pour les appartements.

Question 7. Comparez le prix médian par `type_local` à l'aide d'un `groupby`. Présentez le résultat sous forme de tableau puis réalisez un graphique en barres avec un titre explicite et des axes lisibles.

Piste :

```
df.groupby("type_local")["valeur_fonciere"].agg(["median", "count"])
```

3.2. Dimension temporelle

Question 8. Convertissez `date_mutation` au bon format, extrayez le mois, puis analysez le volume de transactions par mois. Représentez ce volume par une courbe et proposez une hypothèse sur une éventuelle saisonnalité.

Question 9. Étudiez l'évolution mensuelle du prix médian des appartements. Tracez la série temporelle et interprétez-la en quelques lignes.

4. Diagnostic qualité

Vous allez maintenant évaluer le jeu de données sous l'angle de la qualité. Les résultats doivent être justifiés et rédigés clairement.

4.1. Complétude et conformité

Question 10. Calculez le taux de valeurs manquantes pour chaque colonne. Identifiez les colonnes dont le taux dépasse **30%** et expliquez pourquoi cela peut gêner l'analyse.

Question 11. Proposez une stratégie de traitement pour les trois colonnes les plus incomplètes : suppression, imputation ou abandon de la variable. Justifiez vos choix.

4.2. Unicité et cohérence

Question 12. Recherchez les doublons et supprimez-les si nécessaire. Expliquez la clé métier retenue pour définir une transaction unique.

Question 13. Recalculez `prix_m2`, affichez les dix transactions les plus chères au mètre carré, puis filtrez les valeurs aberrantes selon une règle que vous préciserez. Stockez le résultat final dans un DataFrame nommé `df_clean`.

4.3. Profiling automatique

Générez un rapport automatique sur un échantillon du jeu de données nettoyé.

```
from ydata_profiling import ProfileReport

profile = ProfileReport(
    df_clean.sample(5_000, random_state=42),
    title="DVF Nord 2023 - Rapport qualit",
    explorative=True
)

profile.to_file("dvh_report.html")
profile.to_notebook_iframe()
```

Question 14. Le rapport automatique confirme-t-il les problèmes repérés manuellement ? Avez-vous détecté une alerte supplémentaire grâce à cet outil ?

5. Structuration du pipeline

L'objectif est maintenant de réorganiser votre travail sous une forme plus propre et plus proche d'un pipeline de production.

5.1. Fonctions **extract** / **transform** / **load**

Question 15. Écrivez trois fonctions Python :

- `extract(path)` : charge le fichier brut ;
- `transform(df)` : applique les nettoyages et les transformations ;
- `load(df, dest)` : enregistre le résultat au format Parquet.

Exécutez ensuite ces fonctions dans le bon ordre.

Question 16. Dans chaque fonction, ajoutez une trace simple indiquant le nombre de lignes en entrée et en sortie.

5.2. Nature du traitement

Question 17. Votre pipeline correspond-il à un traitement **batch** ou à un traitement **temps réel**? Justifiez votre réponse.

Question 18. Dans quel cas d'usage ce même pipeline devrait-il évoluer vers un traitement en temps réel? Donnez un exemple concret.

5.3. Comparaison CSV / Parquet

Vérifiez expérimentalement l'intérêt du format Parquet.

```
df_clean.to_csv("dvf_clean.csv", index=False)
df_clean.to_parquet("dvf_clean.parquet", compression="snappy")

tcsv = os.path.getsize("dvf_clean.csv") / 1024
tpq = os.path.getsize("dvf_clean.parquet") / 1024
print(f"CSV : {tcsv:,.0f} Ko")
print(f"Parquet : {tpq:,.0f} Ko")
print(f"Gain : x{tcsv / tpq:.1f}")

t0 = time.time(); pd.read_csv("dvf_clean.csv"); lcsv = time.time() - t0
t0 = time.time(); pd.read_parquet("dvf_clean.parquet"); lpq = time.time() - t0
print(f"Lecture CSV : {lcsv:.3f} s")
print(f"Lecture Parquet : {lpq:.3f} s")
```

Question 19. Pourquoi le format Parquet est-il souvent plus rapide à lire que le CSV, même lorsque le fichier est plus petit ?

Question 20. Rechargez les deux fichiers et comparez leurs types de colonnes. Que remarquez-vous, notamment pour `date_mutation` ?

Question 21. Au regard de l'ordre réel des opérations effectuées, votre démarche relève-t-elle davantage d'un **ETL** ou d'un **ELT**? Justifiez précisément.

6. Synthèse rédigée

Cette dernière partie doit être entièrement rédigée en **Markdown** dans le notebook. Elle évalue votre capacité à relier la pratique à une lecture métier des données.

Question 22. Rédigez un court rapport qualité (environ 150 mots) à destination de votre responsable. Organisez-le autour des dimensions suivantes :

- complétude ;

- conformité ;
- unicité ;
- précision.

Pour chaque dimension, donnez au moins :

- un constat appuyé sur un chiffre ;
- une recommandation concrète.

Le texte doit être compréhensible par une personne non technicienne.

Conseils de présentation

- Structurez le notebook avec des titres clairs.
- Commentez vos choix lorsqu'ils ont un impact sur l'analyse.
- Évitez d'enchaîner uniquement du code : un bon notebook explique autant qu'il calcule.
- Soignez la lisibilité des graphiques : titre, axes, unités si nécessaire.
- Vérifiez que le notebook s'exécute du début à la fin sans erreur.